MODULE 13

SOAP API Design with Java

Design, build, and consume enterprise-grade SOAP web services using Java and industry best practices.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 13 OVERVIEW

Many enterprise organizations still rely on SOAP for contract-first design, strong security, and reliable messaging.

- SOAP (Simple Object Access Protocol) is a standards-based protocol for exchanging structured XML information — you'll design secure, contract-first SOAP service contracts in Java.

DURATION & LAB

4 Hours Theory

SOAP, XML, WSDL, XSD, security.

2 Hours Lab

SOAP API Design.

Scenario

Northstar CRM publishes a SOAP contract for a partner.

SOAP USES

XML messages, WSDL contracts, WS-* standards

BEST FOR

Systems that need contract-first design, strong security, and reliability

THE PAYOFF

Strong security, reliability, contract-first design

KEY TAKEAWAY SOAP isn't about being old — it's about being dependable. Right technology, right problem, right environment.

WHY SOAP STILL EXISTS

Despite the rise of REST, SOAP remains critical in enterprise and regulated environments.

Enterprise & Legacy
Mission-critical systems built on SOAP (mainframes, ESBs).

Strong Standards
Mature W3C/OASIS specs ensure interoperability.

Built-In Security
WS-Security: encryption, signatures, tokens.

Reliable & Transactional
WS-ReliableMessaging, WS-AtomicTransaction.

Contract-First
WSDL defines the contract before implementation.

Regulated Industries
Banking, insurance, healthcare, government.

REAL-WORLD EXAMPLES

- Legacy integration: stable SOAP endpoints let modern services talk to mainframes and ESBs without rewriting them.
- Compliance & auditability: WS-Security and WS-ReliableMessaging provide the audit trails and guaranteed delivery regulated processes require.

KEY TAKEAWAY SOAP isn't about being old — it's about being dependable when security, reliability, and compliance matter most.

Why SOAP Still Exists

SOAP VS REST

Two architectural styles for building APIs — they differ in philosophy, protocols, message formats, and use cases.

ASPECT	SOAP	REST
Nature & Protocol	Protocol-based, with strict rules. Typically HTTP, but also SMTP, JMS, TCP	An architectural style. REST APIs are most commonly implemented over HTTP/HTTPS.
Message Format	XML only	JSON, XML, YAML, etc. REST typically layers on HTTP caching (ETags, Cache-Control).
Contract	SOAP commonly uses formal WSDL contracts and often follows a contract-first approach.	REST APIs may be code-first or contract-first using OpenAPI.
Security	Built-in WS-Security; supports enterprise WS-* standards	HTTPS, OAuth 2.0, JWT, API Keys
Performance	SOAP XML messages are generally more verbose.	REST/JSON often produces smaller payloads, but actual performance should be measured.
Java Example	JAX-WS (Jakarta XML Web Services)	JAX-RS (Jakarta RESTful WS), Spring Web

CHOOSE BASED ON CONTRACT NEEDS, SECURITY MODEL, PAYLOAD SIZE, AND TEAM/TOOLING FIT — NOT PROTOCOL ALONE

KEY TAKEAWAY SOAP and REST fit different needs — evaluate contract requirements, security model, and payload characteristics for your use case, and measure performance rather than assuming it.

SOAP VS REST — JAVA EXAMPLE

A JAX-WS service interface generates the WSDL contract the SOAP vs REST comparison refers to — annotations replace hand-written XML.

JAX-WS SERVICE INTERFACE (CODE-FIRST)

```
@WebService(name = "CustomerService",
    targetNamespace = "http://northstar.example.com/customer/v1")
public interface CustomerService {
    @WebMethod(operationName = "GetCustomer")
    @WebResult(name = "customer")
    CustomerType getCustomer(
        @WebParam(name = "customerId") String customerId
    ) throws CustomerFault_Exception;
}
```

JAX-RS marks the REST equivalent with @Path/@GET instead of @WebService/@WebMethod.

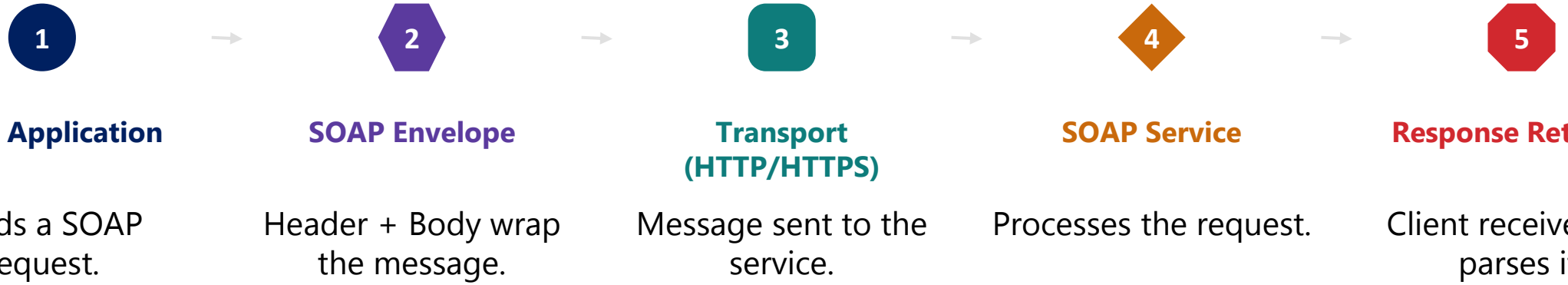
KEY TAKEAWAY JAX-WS (SOAP) and JAX-RS (REST) both map a Java method to a network operation — the contract style they produce is what differs.

SOAP vs REST

Aspect	SOAP	REST
Approach	**Protocol-based** Strict standards and rules	**Architecture style** Guiding principles
Protocol	Uses **SOAP** protocol over HTTP, SMTP, etc.	Uses **HTTP** protocol (GET, POST, PUT, DELETE)
Data Format	**XML** only (Uses XSD for schema)	**JSON, XML, YAML, etc.** (Flexible)
Security	**WS-Security (built-in)** Supports enterprise security standards	**Relies on HTTP security** (OAuth, JWT, HTTPS, etc.)
Standards	**Heavy standards** (WSDL, XSD, SOAP)	**Lightweight** Leverages HTTP and URIs
Complexity	**Complex** More configuration and setup	**Simple** Easy to develop and consume
Performance	**Slower** Large XML payloads	**Faster** Lightweight payloads
Use Cases	**Enterprise systems** Banking, Insurance, Healthcare, Legacy integrations	**Web and mobile apps** Public APIs, Microservices, Cloud applications
Tooling	**Limited and heavier tools** (e.g., SOAP UI)	**Wide range of lightweight tools** (e.g., Postman, Insomnia)
Scalability	**Less scalable** Stateful operations	**Highly scalable** Stateless operations

SOAP ARCHITECTURE OVERVIEW — MESSAGE FLOW

SOAP is a standard protocol for exchanging structured XML information in web services.



KEY TAKEAWAY SOAP provides a robust, standards-based foundation for secure, reliable, interoperable web services.

SOAP ARCHITECTURE OVERVIEW — KEY COMPONENTS

SOAP is a standard protocol for exchanging structured XML information in web services.

KEY COMPONENTS

WSDL

Defines the service contract.

UDDI (Historical)

Historical service-registry standard; rarely used in modern projects.

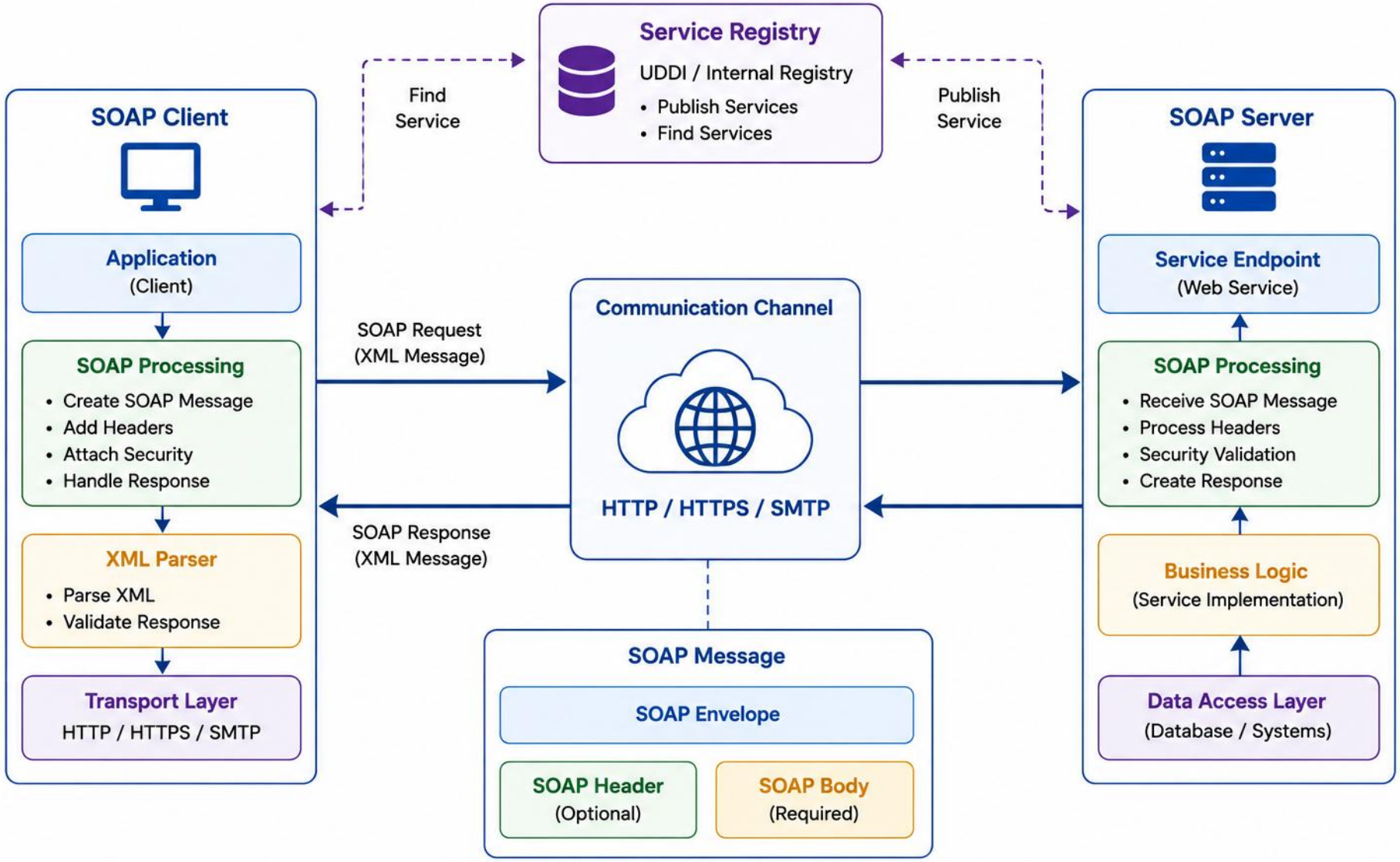
WS-*

Security, reliability, transactions.

WS- EXAMPLES: WS-Security (auth & encryption) • WS-ReliableMessaging (guaranteed delivery) • WS-AtomicTransaction (distributed transactions) • WS-Policy (policy assertions).*

KEY TAKEAWAY SOAP provides a robust, standards-based foundation for secure, reliable, interoperable web services.

SOAP Architecture



XML FUNDAMENTALS

XML is platform-independent, self-descriptive, and extensible — the foundation of SOAP, WSDL, and JAXB.

XML EXAMPLE

```
<?xml version="1.0" encoding="UTF-8"?>
<employee>
  <id>101</id>
  <name>John Doe</name>
  <role>Developer</role>
</employee>
```

XML BASIC RULES

- XML is case-sensitive; every element must have a closing tag.
- Use namespaces (xmlns) for namespace qualification and to avoid collisions.
- Declare character encoding (e.g., UTF-8) and escape special characters.
- Validate against an XSD schema; choose elements vs. attributes deliberately.

COMMON USE CASES

Data Exchange

Namespace Qualification

Web Services (SOAP)

Schema Validation (XSD)

Escape special characters (& < > " ') and use namespaces (xmlns) to avoid name collisions when combining XML from multiple sources.

KEY TAKEAWAY XML provides a flexible, standard way to represent structured data — the foundation for SOAP, WSDL, and JAXB.

XML Fundamentals

What is XML?

XML (eXtensible Markup Language) is a text-based markup language designed to store and transport data.

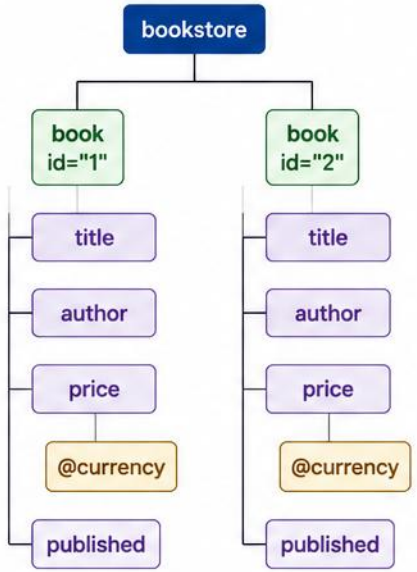
Extensible
Users define their own tags.

Portable
Platform and language independent.

Hierarchical
Organizes data in a tree structure.

XML Structure Example

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
 <book id="1">
    <title>Java Basics</title>
    <author>John Doe</author>
    <price currency="USD">29.99</price>
    <published>true</published>
 </book>
 <book id="2">
    <title>XML Guide</title>
    <author>Jane Smith</author>
    <price currency="USD">39.99</price>
    <published>false</published>
 </book>
</bookstore>
```



Root Element · Parent Element · Child Element · Attribute

Key XML Concepts

Elements
Defined by start and end tags.
`<tag>value</tag>`

Attributes
Provide additional information about elements.
`<tag attr="value">`

Text Content
The actual data inside elements.
`<tag>data</tag>`

Comments
Used to add notes (not displayed).
`<!-- comment -->`

Well-Formed XML Rules

- XML must have a single root element.
- All elements must have proper opening and closing tags.
- Tags are case-sensitive.
- All tags must be properly nested.
- Attribute values must be enclosed in quotes.
- Special characters must be escaped (`<`, `>`, `&`, `&quot;`, `&apos;`).

SOAP MESSAGE STRUCTURE

A SOAP message is an XML document that follows a specific structure — used to exchange information between client and service.

Envelope

- Root element that identifies the message as SOAP.

Header (Optional)

- Metadata: authentication, transactions, routing.

Body (Required)

- Contains the actual request or response data.

Fault (Inside Body)

- Nested inside Body when processing fails — not a sibling of Header/Body.

STRUCTURE RULES

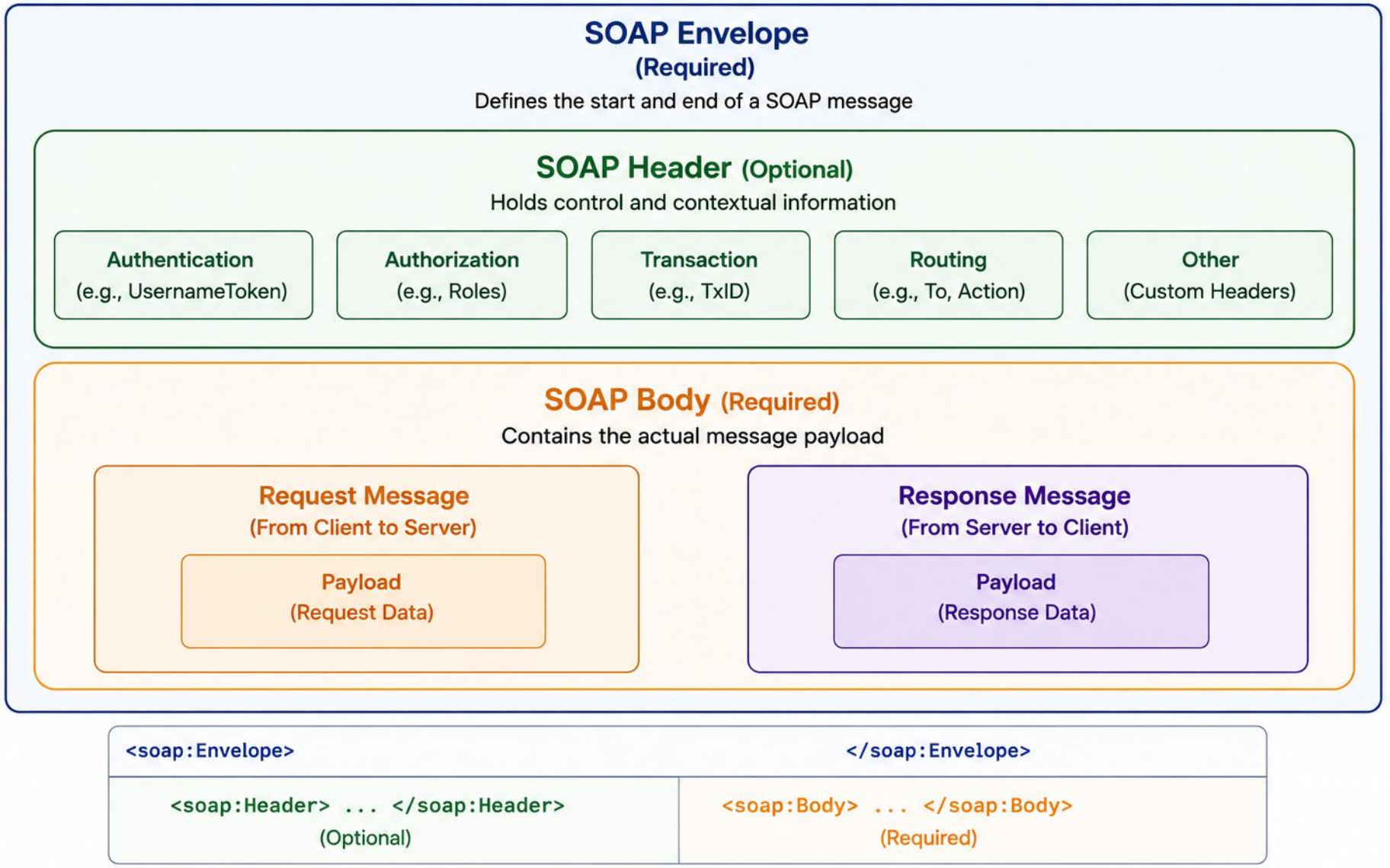
- The message must have a single <soap:Envelope> as the root.
- <soap:Header> is optional and comes before <soap:Body>.
- <soap:Body> is required; a <soap:Fault>, when present, is nested inside Body — not a sibling of Header/Body.

SOAP 1.1 and SOAP 1.2 differ in a few concrete ways:

Aspect	SOAP 1.1	SOAP 1.2
Envelope namespace	schemas.xmlsoap.org/...	www.w3.org/2003/05/...
HTTP content type	text/xml	application/soap+xml
Action	Usually SOAPAction header	Can be content-type parameter
Fault model	faultcode, faultstring, detail	Code, Reason, Detail

KEY TAKEAWAY SOAP messages are XML-based and strictly structured — validate against WSDL schemas and handle faults gracefully.

SOAP Message Structure



SOAP ENVELOPE

The root of every SOAP message — it defines namespaces and wraps Header, Body, and optional Fault.

MINIMAL SOAP ENVELOPE

```
<soap:Envelope xmlns:soap="...envelope/">
  <soap:Header>...</soap:Header>
  <soap:Body>
    <GetDataRequest>...</GetDataRequest>
  </soap:Body>
</soap:Envelope>
```

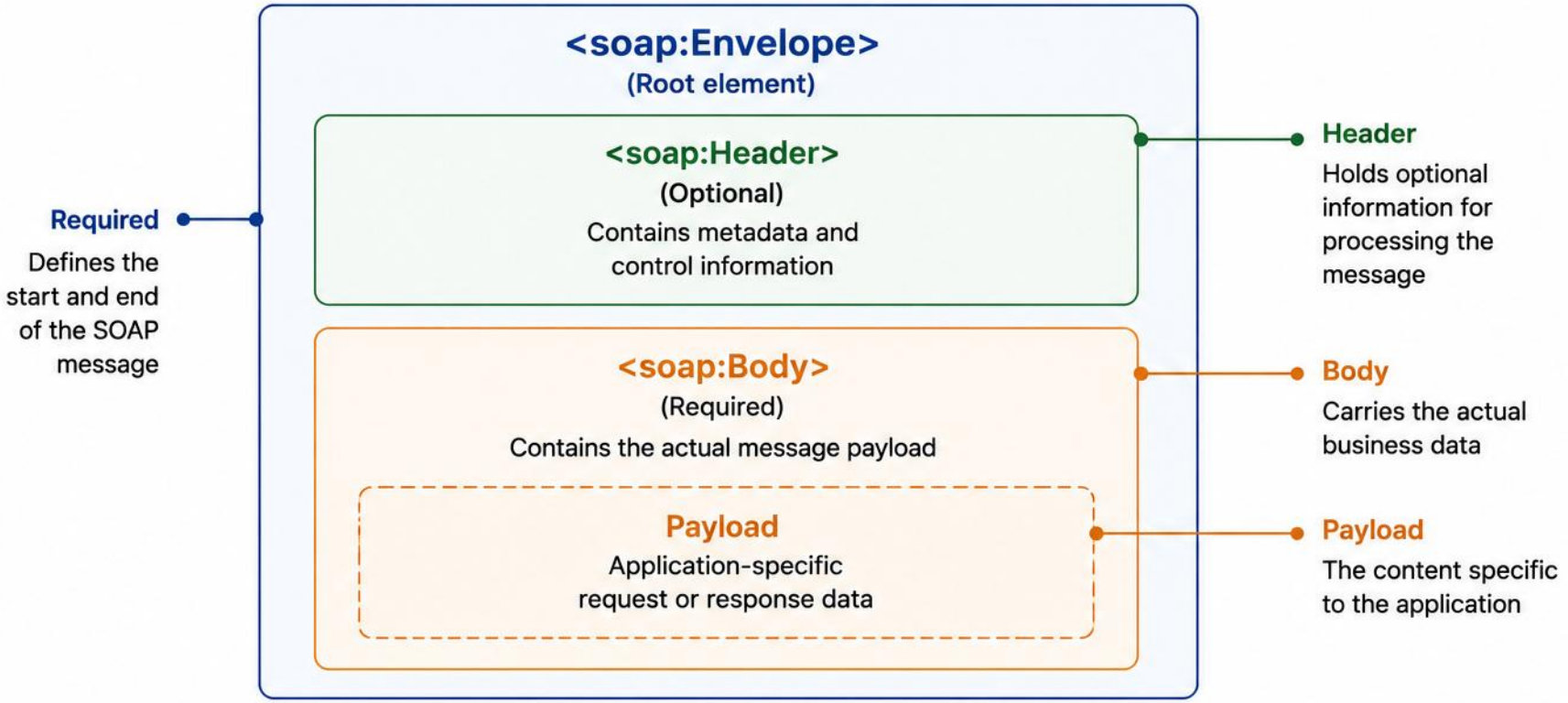
KEY RESPONSIBILITIES

- Defines namespace — declares the SOAP namespace.
- Encapsulates the message — wraps Header, Body, Fault.
- Provides structure — ensures a well-formed message.
- Enables interoperability across compliant systems.

Common namespace prefixes: soap (SOAP 1.1 envelope), soap12 (SOAP 1.2 envelope), xsi (XML Schema instance), xsd (XML Schema).

KEY TAKEAWAY The Envelope is mandatory and must be the first and last element — without it, the message is not valid SOAP.

SOAP Envelope




```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">      <!-- SOAP Envelope Start -->
    <soap:Header>
        <!-- Optional header elements go here -->
    </soap:Header>                                                          <!-- Optional -->
    <soap:Body>
        <!-- Actual message payload goes here -->                           <!-- Required -->
    </soap:Body>
</soap:Envelope>                                                            <!-- SOAP Envelope End -->
```

HEADER, BODY, AND FAULT (1 OF 2)

Header carries optional metadata and Body carries the required payload — the two structural pieces every SOAP message needs.

HEADER — SECURITY EXAMPLE

```
<soap:Header>
  <tns:AuthToken soap:mustUnderstand="1">
    <tns:Token>ABC123TOKEN</tns:Token>
  </tns:AuthToken>
</soap:Header>
```

BODY — REQUEST EXAMPLE

```
<soap:Body>
  <tns:CreateOrderRequest>
    <tns:CustomerId>1001</tns:CustomerId>
  </tns:CreateOrderRequest>
</soap:Body>
```

HEADER — USE CASES

- WS-Security tokens, routing, transactions, and QoS metadata.
- Custom-header example above is simplified — not a complete WS-Security implementation.
- Conceptual: <wsse:Security><wsse:UsernameToken>…</wsse:UsernameToken></wsse:Security>

BODY — KEY POINTS

- Required element in every SOAP message.
- For WS-I document/literal wrapped services, Body normally contains one operation wrapper element — follow the WSDL.
- Defined by the service contract (WSDL); can contain complex data structures.

KEY TAKEAWAY Header carries optional metadata; Body is the one required element in every SOAP message and holds the operation payload.

HEADER, BODY, AND FAULT (2 OF 2)

Fault — nested inside Body, never a sibling of Header — reports errors, plus a few extra attributes worth knowing.

FAULT (INSIDE BODY) — SOAP 1.1 EXAMPLE

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid customer ID</faultstring>

<detail><tns:ValidationFault><tns:code>CUSTOMER_ID_R
EQUIRED</tns:code>
  </tns:ValidationFault></detail>
</soap:Fault>
```

FAULT — KEY POINTS

- SOAP 1.1: faultcode (Client/Server), faultstring, and an optional detail element identify the error.
- SOAP 1.2 uses a different Fault structure: Code, Reason, and Detail instead of faultcode/faultstring/detail.

Key header attributes: mustUnderstand (forces processing or returns a Fault); soap:actor (SOAP 1.1) / soap:role (SOAP 1.2) target the intended recipient node. Body content types include simple elements, complex elements, collections, and custom XSD types.

KEY TAKEAWAY Fault always lives inside Body, never as a sibling of Header — and mustUnderstand/soap:actor control who must process what.

TRY IT YOURSELF — EXERCISE 1: FILL FAULT TODOS (STARTER) (1 OF 2)

Reinforces: the SOAP 1.1 faultcode / faultstring / detail structure you just saw.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — complete a NotFound fault for CUS-9999, not Amina or Ravi
File	notes/lab13-fault-todos.md (in examples/module-13-exercises/)
Fill in	Fault code (Client/NotFound style), fault string, detail customerId

TRY IT NOW Complete Exercise 1 — see exercises/exercise-01-fill-fault-envelope-todos.md (starter).

TRY IT YOURSELF — EXERCISE 1: FILL FAULT TODOS (STARTER) (2 OF 2)

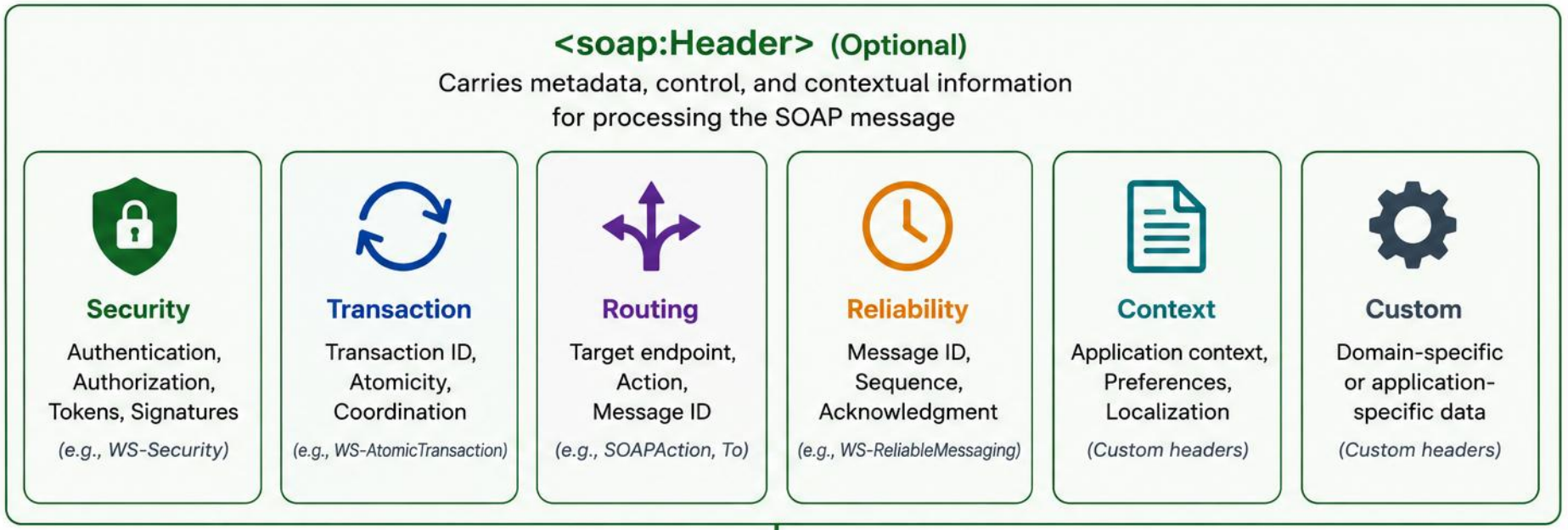
Reinforces: the SOAP 1.1 faultcode / faultstring / detail structure you just saw.

STEP	WHAT TO DO
Fixture	detail customerId = CUS-9999; correlation id = lab-request-001
Common mistake	Using CUS-1001 as the not-found example — keep Amina valid; CUS-9999 is the fault id
Reference	exercises/exercise-01-fill-fault-envelope-todos.md (starter)

```
$ cat lab13-fault-todos.md
Fault code: _____  // Client/NotFound style code
Fault string: _____  // message for unknown customer
Detail customerId: _____  // CUS-9999
Correlation id: _____  // lab-request-001
```

TRY IT NOW Complete Exercise 1 — see exercises/exercise-01-fill-fault-envelope-todos.md (starter).

SOAP Header

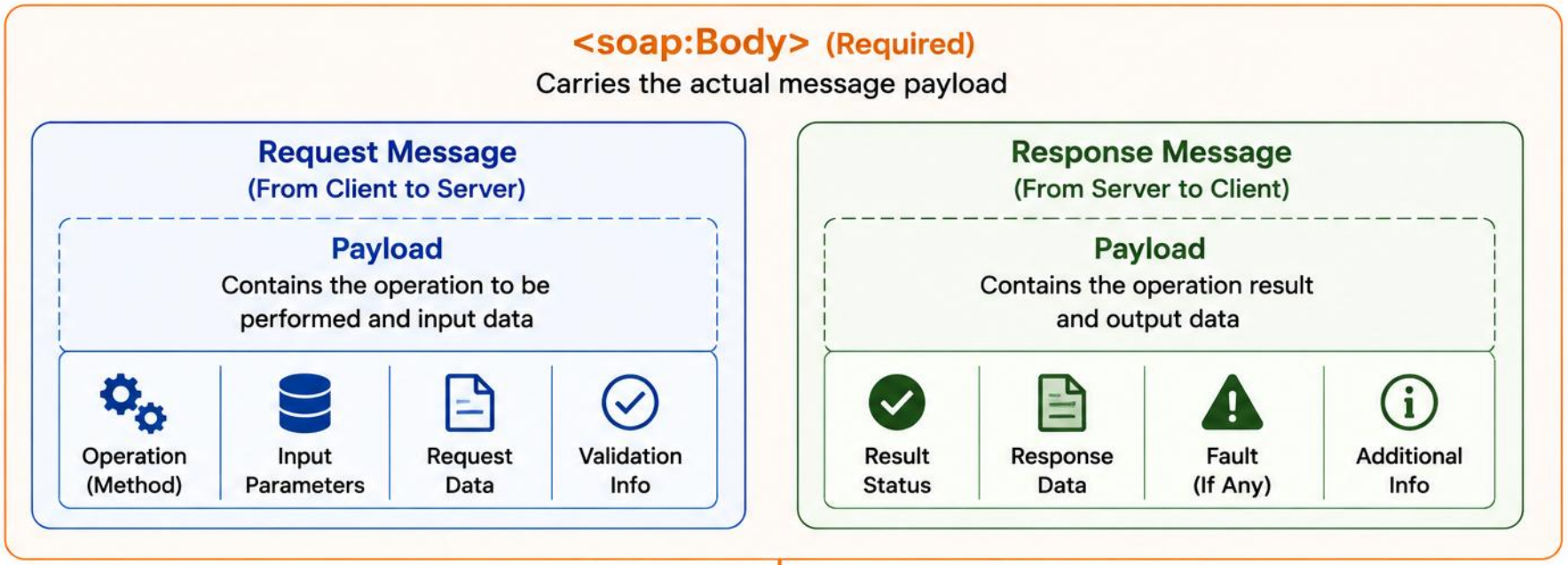



```
<soap:Header  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
              xmlns:wsse="http://docs.oasis-open.org/wss/...">

    <wsse:Security> ...  </wsse:Security>                    <!-- Security information -->
    <wsa:MessageID>uuid:123e4567-e89b...</wsa:MessageID>     <!-- Unique message identifier -->
    <wsa:To>http://service.example.com/endpoint</wsa:To>     <!-- Target endpoint -->
    <wsrm:Sequence> ...  </wsrm:Sequence>                    <!-- Reliability sequence -->
    <app:Context> ... </app:Context>                         <!-- Application context -->
    <app:CustomHeader> ... </app:CustomHeader>               <!-- Custom header -->
</soap:Header>
```

Header Elements

SOAP Body




```
<soap:Body>
   <ns:GetAccountRequest>
      <ns:AccountId>12345</ns:AccountId>
      <ns:IncludeDetails>true</ns:IncludeDetails>
   </ns:GetAccountRequest>
   <ns:GetAccountResponse>
      <ns:Status>SUCCESS</ns:Status>
      <ns:Account>
         <ns:AccountId>12345</ns:AccountId>
         <ns:Balance>2500.75</ns:Balance>
      </ns:Account>
   </ns:GetAccountResponse>
</soap:Body>
```

Request Payload
Operation + Input Data

Response Payload
Result + Output Data

WSDL OVERVIEW

WSDL is an XML-based language that describes what a SOAP service does, how to access it, and what messages it exchanges.

WHAT IS WSDL?

- WSDL is an XML document that describes a web service.
- It acts as a contract between provider and consumer.
- It does not contain the implementation, only the interface.
- Used by tools to generate client proxies and server skeletons.

WSDL FILE STRUCTURE

```
<definitions>
  <types>...</types>
  <message>...</message>
  <portType>...</portType>
  <binding>...</binding>
  <service>...</service>
</definitions>
```

WSDL VERSIONS & VERSIONING STRATEGY

- WSDL 1.1 remains the most common version in enterprise SOAP tooling. WSDL 2.0 exists but has much lower adoption and tool support.
- Prefer document/literal wrapped contracts for broad interoperability unless an existing integration requires another style.
- Other service description languages: OpenAPI/Swagger and RAML describe REST APIs; Protocol Buffers describe gRPC services.
- Versioning strategy: bump the target namespace (e.g., .../customer/v2) or endpoint path for breaking changes; add new fields as optional to stay backward-compatible; keep prior versions live until consumers migrate.

KEY TAKEAWAY WSDL is the blueprint of a SOAP web service — it enables interoperability, automation, and clear communication.

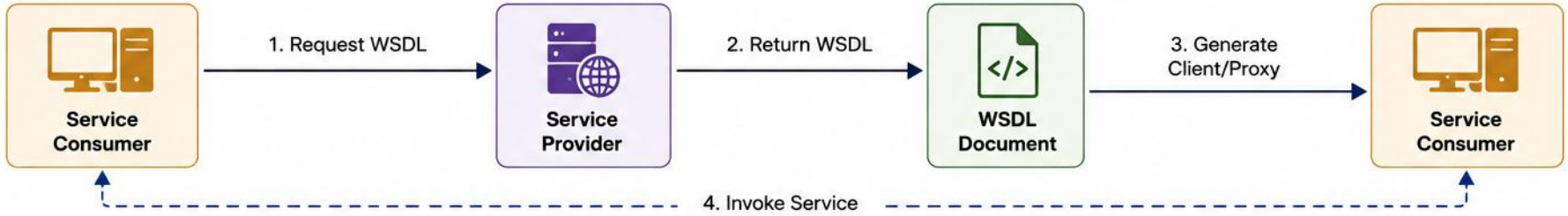
WSDL Overview

What is WSDL?

WSDL (Web Services Description Language) is an XML-based contract that describes a web service's functionality, endpoints, and data types.

- Contract-First Development
- Technology Independent
- Platform Independent
- Standardized Integration

How WSDL Works



WSDL Structure

<definitions>

- **<types>** — Defines the data types (XSD Schemas)
- **<message>** — Defines the input and output messages
- **<portType>** — Defines the operations and message exchange
- **<binding>** — Specifies the protocol and data format
- **<service>** — Defines the service endpoint (URL)

WSDL Components

Types	Defines the XML Schema data types used.
Messages	Defines the input and output message parts.
PortType	Defines the operations (abstract interface).
Binding	Binds portType to a concrete protocol and data format.
Service	Defines the endpoint address where the service is available.

WSDL Example (Snippet)

```
<definitions name="OrderService"
   targetNamespace="http://example.com/order"
   xmlns="http://schemas.xmlsoap.org/wsdl/"
   xmlns:tns="http://example.com/order"
   xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <types>...</types>
  <message name="GetOrderRequest">...</message>
  <message name="GetOrderResponse">...</message>
  <portType name="OrderPortType">...</portType>
  <binding name="OrderBinding" type="tns:OrderPortType">
    ...
  </binding>
  <service name="OrderService">
    <port name="OrderPort" binding="tns:OrderBinding">
        <soap:address location="http://example.com/order"/>
    </port>
  </service>
</definitions>
```

WSDL COMPONENTS

Each WSDL component has a specific role in describing how the service works and how clients interact with it.

COMPONENT	PURPOSE	DEPENDS ON
`definitions`	Root element that wraps the entire WSDL document	None (root)
`types`	Defines data types used in messages (XSD)	definitions
`message`	Defines the data being sent or received	types
`portType`	Defines a set of operations supported	message
`binding`	Binds a portType to a protocol (SOAP) and format	portType
`service`	Defines the endpoint address (URL)	binding

ABSTRACT LAYER (TYPES, MESSAGE, PORTTYPE) DEFINES WHAT THE SERVICE DOES; CONCRETE LAYER (BINDING, SERVICE) DEFINES HOW AND WHERE IT'S ACCESSED.

KEY TAKEAWAY WSDL is a contract-first approach — each component has a specific responsibility, together describing the full service.

WSDL COMPONENTS — EXAMPLE

One condensed WSDL fragment showing how types, message, portType, binding, and service chain together.

CustomerService.wsdl (CONDENSED)

```
<definitions targetNamespace="http://northstar.../customer/v1">
  <types>...CustomerType (see XSD)...</types>
  <message name="GetCustomerRequest">...</message>
  <portType name="CustomerPortType">
    <operation name="GetCustomer">
      <input message="tns:GetCustomerRequest"/>
      <output message="..Response"/></operation></portType>
  <binding name="CustomerBinding" type="tns:CustomerPortType">...</binding>
  <service name="CustomerService">...</service></definitions>
```

Abstract layer (types, message, portType) defines the contract; concrete layer (binding, service) wires it up.

KEY TAKEAWAY Every WSDL component from the previous slide shows up here in one contract — trace how each depends on the one before it.

TRY IT YOURSELF — EXERCISE 2: OPERATION MATRIX (1 OF 2)

Reinforces: portType — the WSDL component that defines a set of operations supported.

STEP	WHAT TO DO
Goal	Fill an in / out / fault matrix for GetCustomer and ActivateCustomer
File	notes/lab13-operation-matrix.md (in examples/module-13-exercises/)
GetCustomer	In: customerId; Out: id, name, status; Fault: not found

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-operation-matrix.md.

TRY IT YOURSELF — EXERCISE 2: OPERATION MATRIX (2 OF 2)

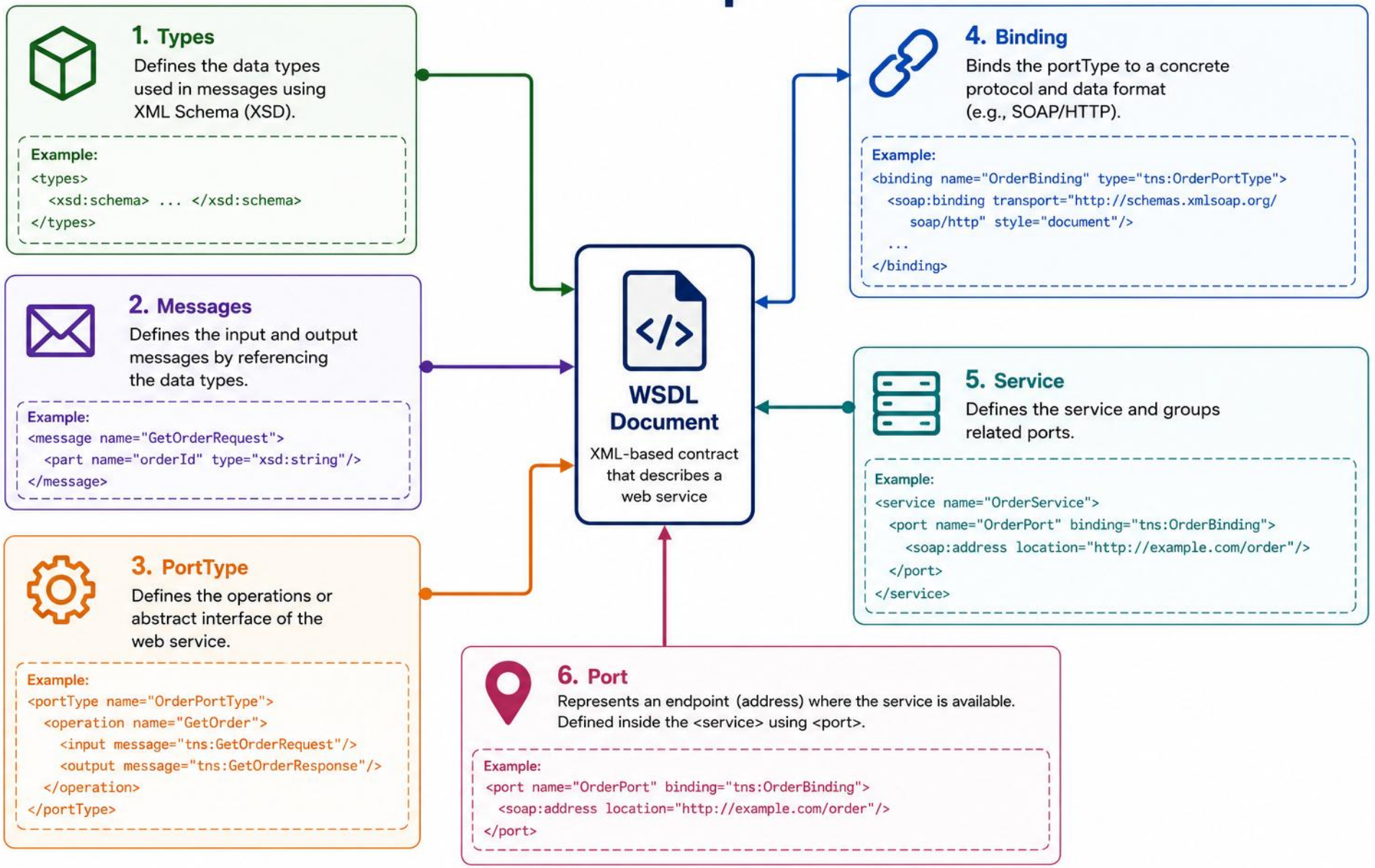
Reinforces: portType — the WSDL component that defines a set of operations supported.

STEP	WHAT TO DO
ActivateCustomer	In: customerId (+ correlation header idea); Out: new status; Fault: invalid transition
Happy path	Activate CUS-1002 Ravi PROSPECT → ACTIVE is the design happy path
Reference	exercises/exercise-02-operation-matrix.md

```
$ cat lab13-operation-matrix.md
GetCustomer       In: customerId
                  Out: id, name, status  |  Fault: not found
ActivateCustomer  In: customerId (+ correlation header idea)
                  Out: new status  |  Fault: invalid transition  // design only
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-operation-matrix.md.

WSDL Components



XSD SCHEMA DESIGN — PRINCIPLES & INDICATORS

XML Schema (XSD) defines the structure, data types, constraints, and relationships of XML documents.

XSD DESIGN PRINCIPLES

- Reusable complexType and simpleType definitions for shared structures.
- xs:sequence controls child-element order within a complexType.
- Constrain values: enumerations, pattern restrictions, length limits.
- Use elementFormDefault="qualified"; design namespaces deliberately.
- Choose correct data types; use meaningful, consistent names.

OCCURRENCE INDICATORS

VALUE	MEANING
`minOccurs="0"`	Element is optional
`maxOccurs="1" (default)`	Element appears once
`maxOccurs="unbounded"`	Element can repeat
`nillable="true"`	PRESENT with xsi:nil="true" — different from minOccurs="0" (ABSENT)

KEY TAKEAWAY Well-designed XSD schemas ensure data quality, consistency, and interoperability — validate early and often.

XSD SCHEMA DESIGN — EXAMPLE

customer.xsd showing complexType, simpleType restriction, sequence, and occurrence indicators together.

customer.xsd (EXCERPT)

```
<xs:complexType name="CustomerType">
  <xs:sequence>
    <xs:element name="customerId" type="xs:string"/>
    <xs:element name="status" type="tns:CustomerStatus"/>
    <xs:element name="notes" type="xs:string" minOccurs="0"/>
  </xs:sequence></xs:complexType>
<xs:simpleType name="CustomerStatus">
  <xs:restriction base="xs:string">
    <xs:enumeration value="PROSPECT"/>
    <xs:enumeration value="ACTIVE"/></xs:restriction></xs:simpleType>
```

minOccurs="0" makes notes optional; the enumeration restricts CustomerStatus to two values.

KEY TAKEAWAY complexType + sequence define shape; simpleType + restriction constrain values — together they turn XSD principles into a validated contract.

XSD SCHEMA DESIGN — WHY IT MATTERS

XML Schema (XSD) defines the structure, data types, constraints, and relationships of XML documents.

WHY DESIGN A GOOD XSD SCHEMA?

Data Validation

Ensures XML conforms to rules.

Reusability

Schemas reused across apps.

Interoperability

Common contract for exchange.

KEY TAKEAWAY Well-designed XSD schemas ensure data quality, consistency, and interoperability — validate early and often.

TRY IT YOURSELF — EXERCISE 3: JAVA TO XSD MAP (1 OF 2)

Reinforces: choosing correct XSD data types for Customer fixtures.

STEP	WHAT TO DO
`Goal`	Map Customer fields to XSD-friendly types; add Ravi Singh PROSPECT as a second row set
`File`	notes/lab13-java-xsd-map.md (in examples/module-13-exercises/)
`Amina row`	customerId → xsd:string (CUS-1001); fullName → xsd:string (Amina Khan); status → ACTIVE

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-java-xsd-map.md.

TRY IT YOURSELF — EXERCISE 3: JAVA TO XSD MAP (2 OF 2)

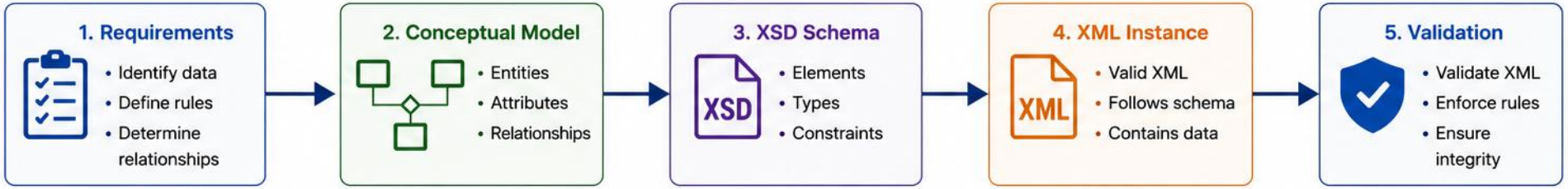
Reinforces: choosing correct XSD data types for Customer fixtures.

STEP	WHAT TO DO
Ravi row	customerId → xsd:string (CUS-1002); fullName → xsd:string (Ravi Singh); status → PROSPECT
Id pattern	Propose CUS-#### as a documentation pattern — not enforced in code yet
Reference	exercises/exercise-03-java-xsd-map.md

```
$ cat lab13-java-xsd-map.md
Java idea            XSD idea      Example
String customerId    xsd:string    CUS-1001
String fullName       xsd:string    Amina Khan
enum/status           xsd:string    ACTIVE  // mapping on paper != generated JAXB
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-java-xsd-map.md.

XSD Schema Design



XSD Schema Structure

```
<xs:schema>
  <xs:element>
  Defines root and child elements
  <xs:complexType>
  Defines complex data types
  <xs:simpleType>
  Defines simple data types
  <xs:attribute>
  Defines attributes
  <xs:restriction> / <xs:enumeration>
  Defines constraints and enumerations
</xs:schema>
```

XSD Building Blocks

Component	Purpose
Element	Defines data structures and hierarchy
ComplexType	Groups elements and/or attributes
SimpleType	Defines primitive data with constraints
Attribute	Provides additional information
Restriction	Restricts values based on rules (length, pattern, etc.)
Enumeration	Defines a list of allowed values

XSD Schema Example

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="Customer">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="Id" type="xs:int"/>
        <xs:element name="Name" type="xs:string"/>
        <xs:element name="Email" type="xs:string"/>
      </xs:sequence>
      <xs:attribute name="status">
        <xs:simpleType>
          <xs:restriction base="xs:string">
            <xs:enumeration value="Active"/>
            <xs:enumeration value="Inactive"/>
          </xs:restriction>
        </xs:simpleType>
      </xs:attribute>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Common Data Types

xs:string | xs:int | xs:decimal | xs:date | xs:boolean | xs:anyType

Common Constraints

minLength | maxLength | pattern | minInclusive | maxInclusive | enumeration

CONTRACT-FIRST DESIGN

Contract-first and code-first are both valid approaches — choose based on the integration's needs.

CONTRACT-FIRST

- Best for shared external contracts
- Strong control over XML and namespaces
- Better consumer/provider coordination
- Requires schema-design skills
- Good for long-lived integrations

VS

CODE-FIRST

- Useful for internal/simple services
- Faster initial implementation
- Contract generated from code
- May expose implementation details
- Good for prototypes or controlled consumers

KEY TAKEAWAY Contract-first suits shared, long-lived integrations; code-first suits internal or fast-moving services — pick based on your consumers and constraints.

CONTRACT-FIRST DESIGN — IN CODE

The same Customer type, designed contract-first in XSD versus generated code-first from an annotated Java class.

CONTRACT-FIRST — XSD DRIVES THE JAVA

```
<xs:complexType name="CustomerType">
  <xs:sequence>
    <xs:element name="customerId" type="xs:string"/>
    <xs:element name="status" type="tns:CustomerStatus"/>
  </xs:sequence></xs:complexType>  <!-- xjc generates Java -->
```

CODE-FIRST — JAVA DRIVES THE XSD (RISK: LEAKAGE)

```
@XmlAccessorType(XmlAccessType.FIELD)
public class CustomerType {
    private String customerId;
    private CustomerStatus status;  // rename = drift
}
```

KEY TAKEAWAY Contract-first keeps the XSD authoritative and generates Java from it; code-first risks framework types and refactors leaking into the contract.

TRY IT YOURSELF — EXERCISE 4: CONTRACT-FIRST MINDSET (1 OF 2)

Reinforces: why contract-first suits shared, long-lived integrations you just saw.

STEP	WHAT TO DO
Goal	Explain why Northstar SOAP should start from contract, not Java classes
File	notes/lab13-contract-first.md (in examples/module-13-exercises/)
Definition	One sentence: define types and operations in XSD/WSDL before generating Java

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-contract-first.md.

TRY IT YOURSELF — EXERCISE 4: CONTRACT-FIRST MINDSET (2 OF 2)

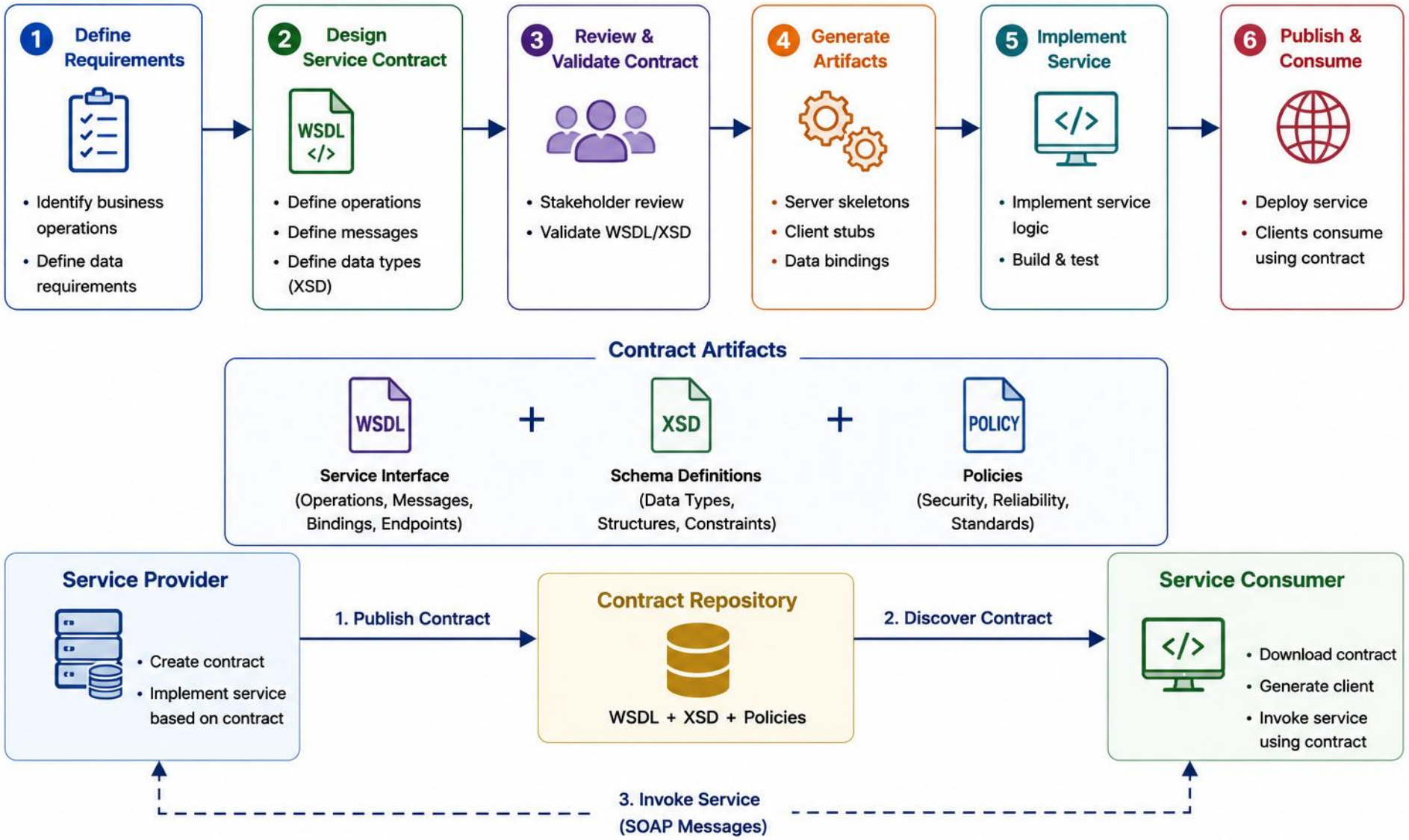
Reinforces: why contract-first suits shared, long-lived integrations you just saw.

STEP	WHAT TO DO
Code-first risks	Name two: accidental breaking changes; framework leakage into the contract
CRM ops	List candidate ops (paper names only): GetCustomer, ActivateCustomer
Reference	exercises/exercise-04-contract-first.md

```
$ cat lab13-contract-first.md
Contract-first: XSD/WSDL types and operations before Java
Risk 1: accidental breaking changes  // code-first
Risk 2: framework leakage into the contract  // code-first
Candidate ops: GetCustomer, ActivateCustomer  // paper names only
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-contract-first.md.

Contract-First Design



SOAP REQUEST–RESPONSE LIFECYCLE — MESSAGE FLOW

A SOAP message travels from client to service and back through seven well-defined steps.

1	2	3	4	5	6	7
Marshal to XML	**Create Envelope**	**Apply Headers**	**Send**	**Validate & Process**	**Generate Result**	**Unmarshal**
Client marshals the request object to XML.	SOAP Envelope is created.	Headers and security are applied.	Message is sent over transport.	Service validates and processes it.	Response or Fault is generated.	Client unmarshals the result.

KEY TAKEAWAY The SOAP request-response lifecycle moves through marshaling, enveloping, transport, processing, and unmarshaling — with the SOAP Fault carrying service-level errors.

SOAP REQUEST-RESPONSE LIFECYCLE — CODE EXAMPLE

Marshaling a request object to XML and unmarshaling the response — the client-side code behind steps 1 and 7.

JAXB MARSHAL / UNMARSHAL (CLIENT-SIDE)

```
JAXBContext ctx = JAXBContext.newInstance(GetCustomerRequest.class);
Marshaller marshaller = ctx.createMarshaller();
GetCustomerRequest request = new GetCustomerRequest();
request.setCustomerId("CUS-1002");
StringWriter xml = new StringWriter();
marshaller.marshal(request, xml);
// xml.toString() is now the <GetCustomerRequest> XML
Unmarshaller unmarshaller = ctx.createUnmarshaller();
GetCustomerResponse response = (GetCustomerResponse)
    unmarshaller.unmarshal(new StringReader(responseXml));
```

This client-side marshal/unmarshal pair is separate from hosting a service — no endpoint is created here.

KEY TAKEAWAY Marshal turns a Java object into the XML the Envelope will carry; unmarshal reverses it on the way back — steps 1 and 7 of the lifecycle.

SOAP REQUEST–RESPONSE LIFECYCLE — OUTCOMES & PRACTICES

A SOAP message travels from client to service and back through seven well-defined steps.

BEST PRACTICES

- Always use HTTPS for secure communication.
- Validate requests at the service boundary.
- Keep the SOAP request size reasonable; handle timeouts gracefully.

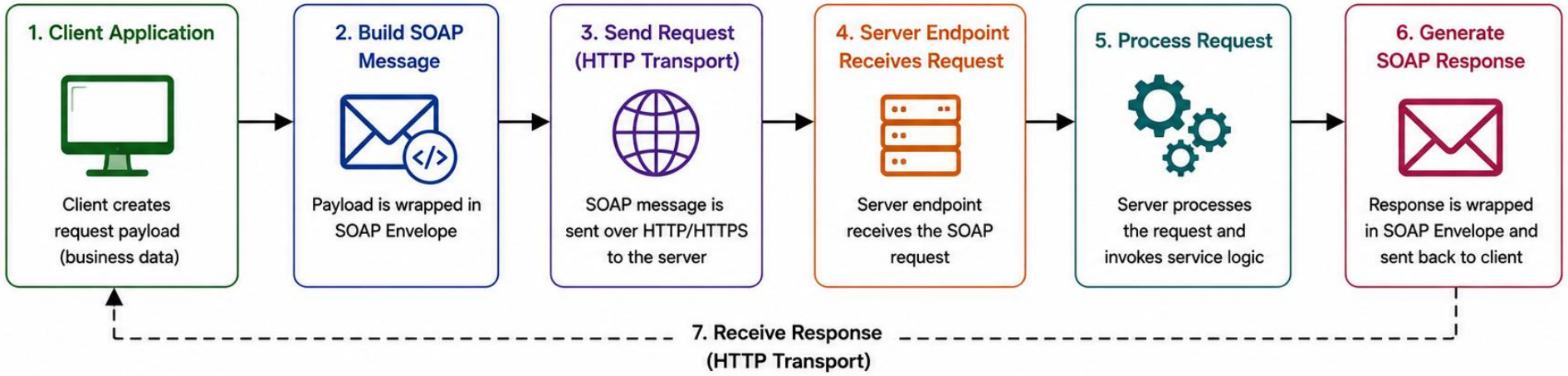
TYPICAL HTTP OUTCOMES

Situation	Typical HTTP outcome
Successful SOAP response	200 OK
Authentication rejected at HTTP layer	401 or 403
Malformed request / SOAP 1.2 sender fault	Often 400
SOAP processing/server fault	Often 500

The SOAP Fault payload is the primary service-level error contract; HTTP status usage varies by version and framework.

KEY TAKEAWAY The SOAP request-response lifecycle moves through marshaling, enveloping, transport, processing, and unmarshaling — with the SOAP Fault carrying service-level errors.

SOAP Request Flow



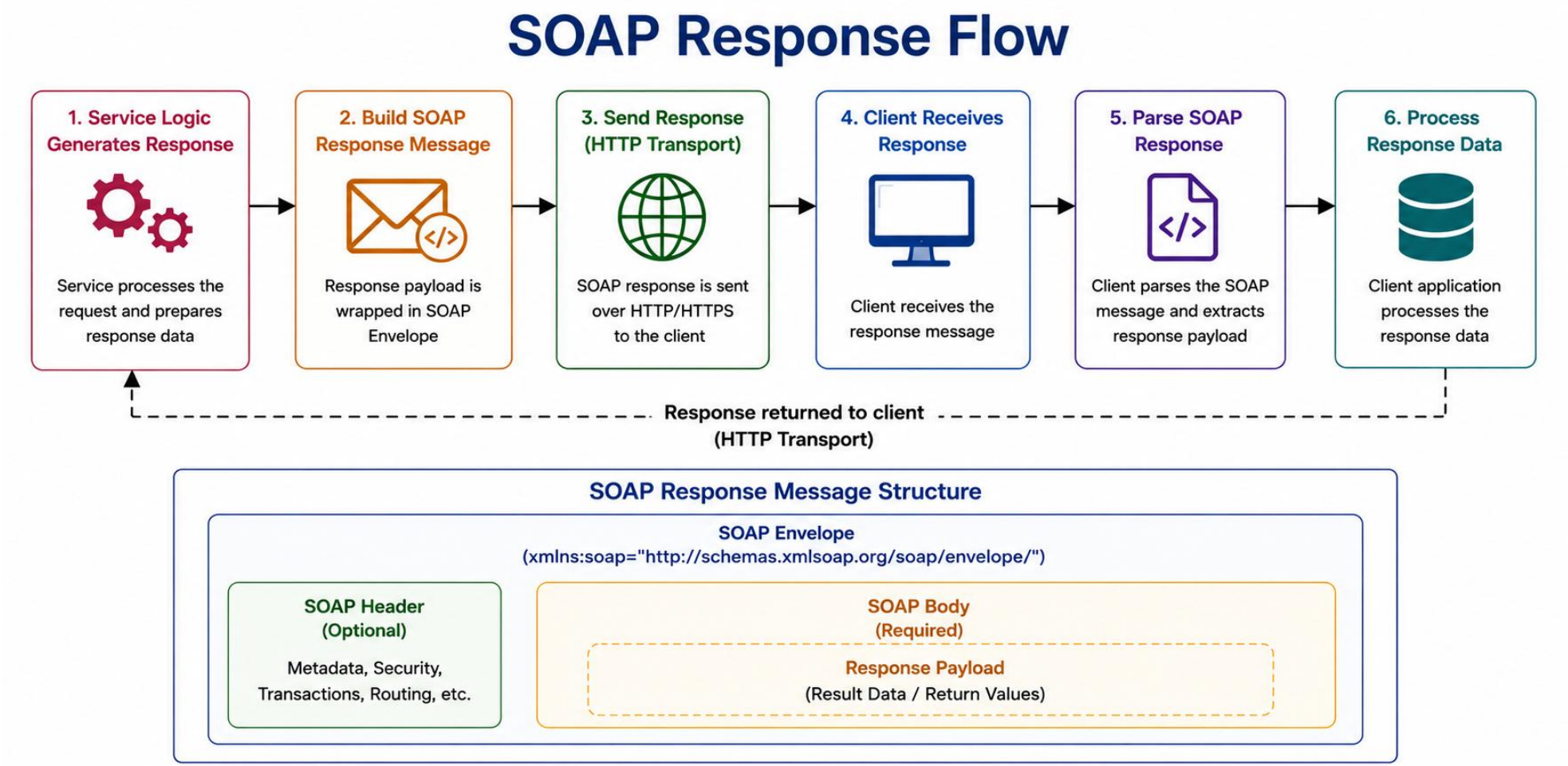
SOAP Request Message Structure

SOAP Envelope
(xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/")

SOAP Header
(Optional)
Metadata, Security, Routing, etc.

SOAP Body
(Required)

Request Payload
Business Data / Operation Parameters



ENTERPRISE SOAP USE CASES

SOAP Web Services continue to power mission-critical enterprise integrations where reliability and security are non-negotiable.

Financial Services

Core banking host integration, payment gateway transactions, and settlement systems.

Healthcare

Healthcare partner contracts for patient, lab, and billing data exchange (HIPAA).

Insurance

Insurance policy and claims exchange across agents, carriers, and adjudication systems.

Retail & E-commerce

ERP and supply-chain integrations for order, inventory, and fulfillment data.

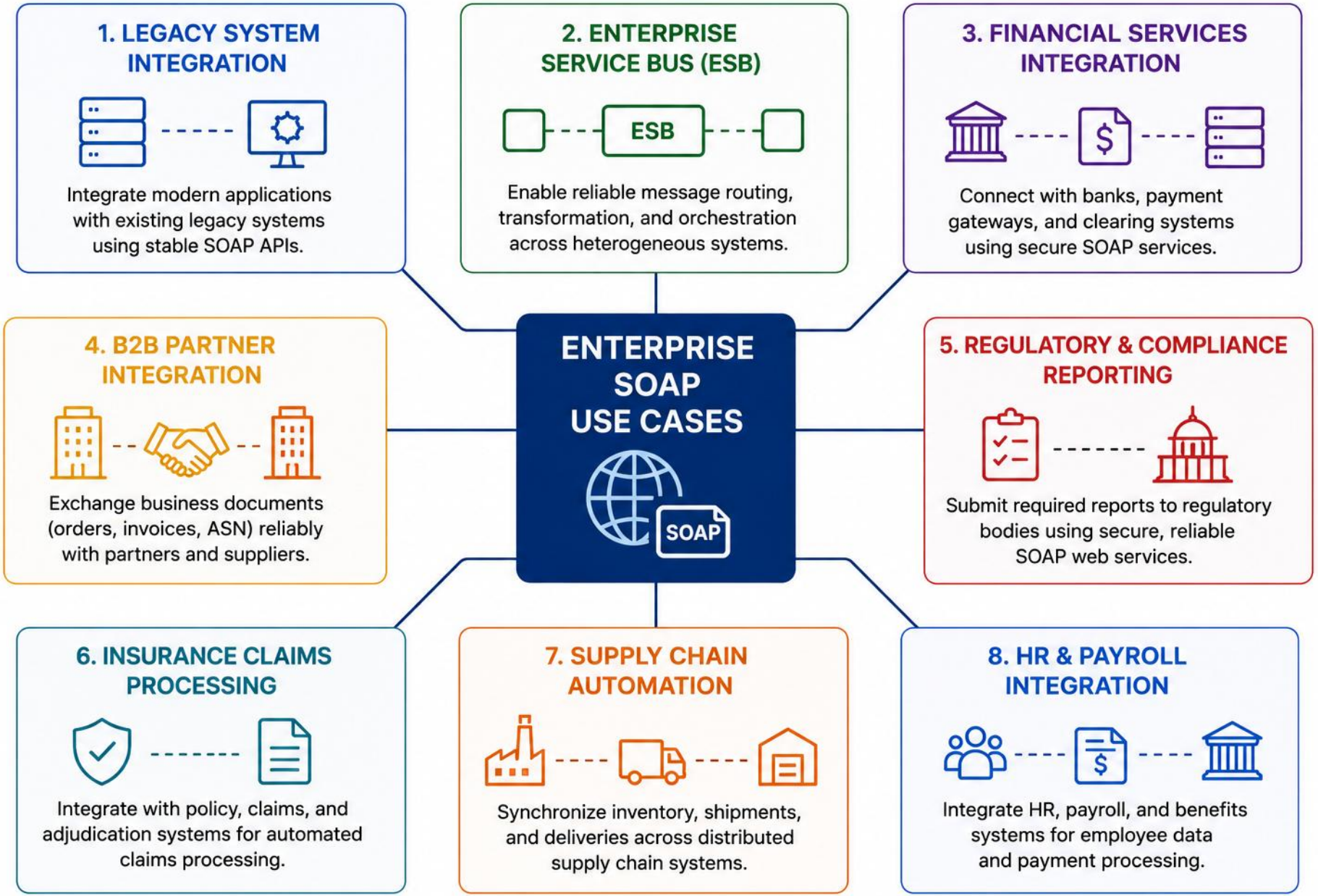
Government

Government B2B interfaces for secure cross-department and citizen-service data exchange.

Manufacturing

Legacy ESB integration linking plant systems with supply-chain and partner networks.

KEY TAKEAWAY SOAP remains a trusted choice for enterprise integrations demanding security, reliability, and standards.



SECURITY CONSIDERATIONS — LAYERED DEFENSES

SOAP services exchange critical business data — security must be built in at every layer: transport, message, and service.

Transport Layer

- Use HTTPS with currently approved TLS versions and cipher suites per organizational policy.

Message Layer (WS-Security)

- UsernameToken, X.509 certificates, XML Signature; guard against XML Signature Wrapping and replay (timestamps, nonces).

Confidentiality

- XML Encryption for sensitive elements.

Service Layer

- Role-based access control, input validation, and schema validation before processing.

XML Parser Hardening

- Disable external entities (XXE); limit entity expansion, payload size, and nesting depth.

Logging & Monitoring

- Log metadata, correlation IDs, operation names, status, and safe diagnostic context. Redact or avoid sensitive message content.

KEY TAKEAWAY Security in SOAP is multi-layered — secure the transport, protect the message, harden the service, monitor continuously.

SECURITY CONSIDERATIONS — CODE EXAMPLE

A WS-Security UsernameToken on the wire, and the parser setting that keeps XML parsing from becoming an XXE hole.

MESSAGE LAYER — WS-SECURITY USERNAMETOKEN

```
<wsse:Security soap:mustUnderstand="1">
  <wsse:UsernameToken>
    <wsse:Username>svc-northstar</wsse:Username>
    <wsse:Password Type="...#PasswordDigest">dGhpc0lz...=</wsse:Password>
  </wsse:UsernameToken></wsse:Security>
```

XML PARSER HARDENING — DISABLE XXE

```
DocumentBuilderFactory dbf = DocumentBuilderFactory.newInstance();
dbf.setFeature(
    "http://apache.org/xml/features/disallow-doctype-decl", true);
dbf.setXIncludeAware(false);
dbf.setExpandEntityReferences(false);   // blocks XXE injection
```

KEY TAKEAWAY WS-Security tokens authenticate the caller; parser hardening (disallow DOCTYPE, disable entity expansion) closes the XXE hole the same XML parsers open.

SECURITY CONSIDERATIONS — WS-SECURITY: WHAT PROTECTS WHAT

SOAP services exchange critical business data — security must be built in at every layer: transport, message, and service.

WS-SECURITY: WHAT PROTECTS WHAT

Authentication tokens (e.g., UsernameToken) — identify the caller.

XML Signature — protects message integrity and authenticity.

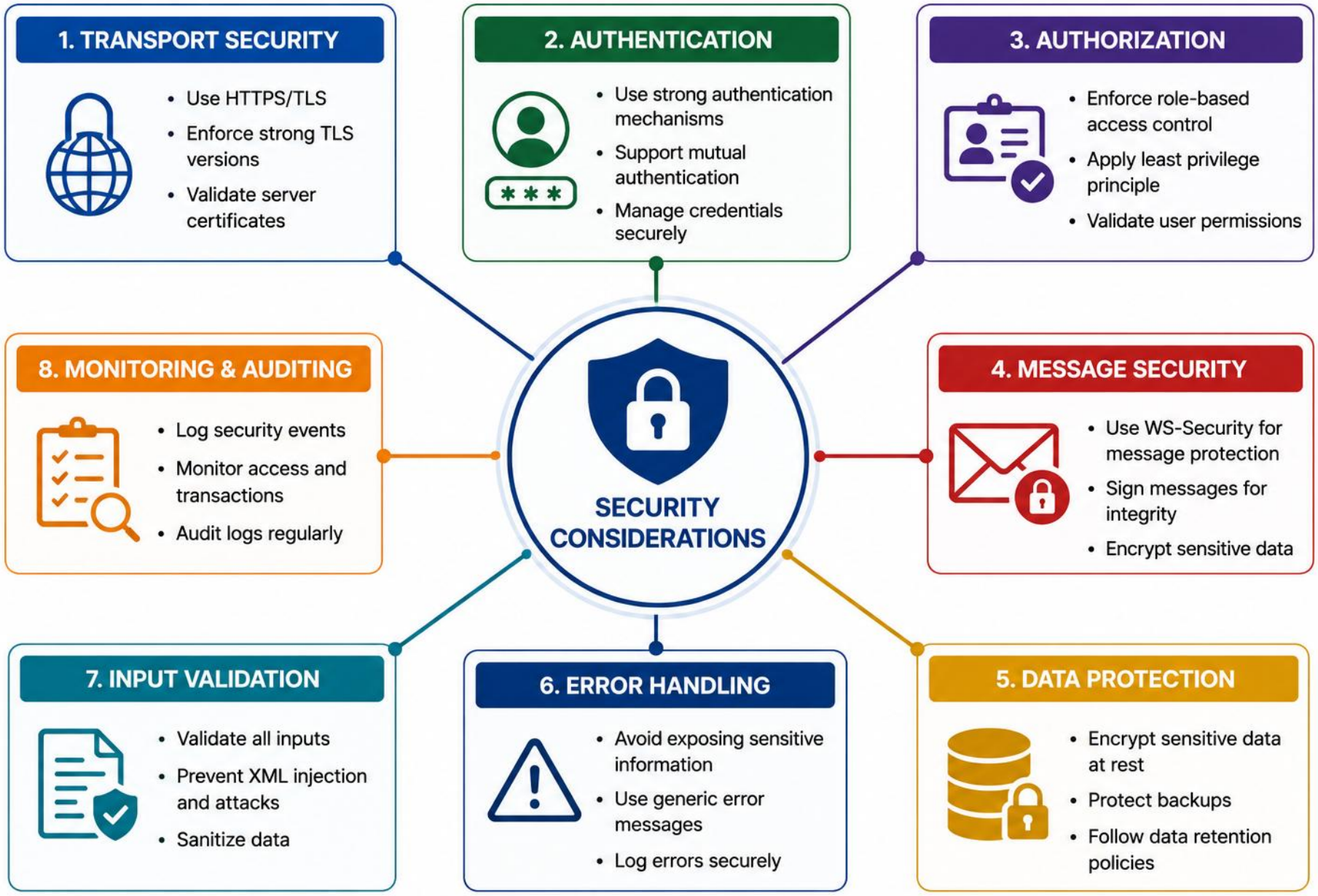
XML Encryption — protects message-level confidentiality.

Timestamps / unique message IDs — prevent replay.

HTTPS/TLS — protects the transport.

WS-Security — can protect the message end-to-end, independent of transport.

KEY TAKEAWAY Security in SOAP is multi-layered — secure the transport, protect the message, harden the service, monitor continuously.



TRY IT YOURSELF — EXERCISE 5: PLACEHOLDER ENDPOINT HONESTY (1 OF 2)

Reinforces: no live server required — design-time validation only, before this lab's scope.

STEP	WHAT TO DO
Goal	Document what a placeholder endpoint means before Spring Boot hosting
File	notes/lab13-placeholder-honesty.md (in examples/module-13-exercises/)
Define	One sentence: contract + sample messages, without a production-ready host

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-placeholder-endpoint-honesty.md.

TRY IT YOURSELF — EXERCISE 5: PLACEHOLDER ENDPOINT HONESTY (2 OF 2)

Reinforces: no live server required — design-time validation only, before this lab's scope.

STEP	WHAT TO DO
Not doing	No Spring-WS @Endpoint, no Boot app, no deploy to Tomcat in this prep
What Lab 24 adds	Lab 24 introduces Spring hosting for this same SOAP contract
Reference	exercises/exercise-05-placeholder-endpoint-honesty.md

```
$ cat lab13-placeholder-honesty.md
Placeholder = contract + sample messages, no production host
Not doing: @Endpoint, Boot app, Tomcat deploy  // prep scope
Lab 24 adds: Spring-WS hosting for this contract  // hosting later
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-placeholder-endpoint-honesty.md.

TRY IT YOURSELF — EXERCISE 6: LAB 13 PREP CHECKLIST (1 OF 2)

Capstone: confirm SOAP design prep is complete before Lab 13 begins.

STEP	WHAT TO DO
Goal	Confirm SOAP design prep without completing Lab 13
File	notes/lab13-prep-checklist.md (in examples/module-13-exercises/)
Artifacts	Confirm contract-first note, XSD map, operation matrix, and fault TODOs all exist

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab13-prep-checklist.md.

TRY IT YOURSELF — EXERCISE 6: LAB 13 PREP CHECKLIST (2 OF 2)

Capstone: confirm SOAP design prep is complete before Lab 13 begins.

STEP	WHAT TO DO
Fixtures	Recall Amina ACTIVE, Ravi PROSPECT, fault id CUS-9999
Pass / Fail	Pass if the placeholder honesty note exists; else revisit Exercise 5
Reference	exercises/exercise-06-lab13-prep-checklist.md

```
$ ls notes/
lab13-contract-first.md      lab13-java-xsd-map.md
lab13-operation-matrix.md    lab13-fault-todos.md
lab13-placeholder-honesty.md  // all five must exist before Lab 13
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab13-prep-checklist.md.

LAB OVERVIEW – SOAP API DESIGN

Design a contract-first SOAP interface (WSDL + XSD) for Northstar's CRM customer service — no server required.

BUSINESS SCENARIO

- Northstar's CRM must publish a stable SOAP contract before a billing partner integrates — architects freeze the XML and operations before Lab 24 builds the endpoint.
- Operations: CreateCustomer, UpdateCustomer, GetCustomer.

SCOPE & CONSTRAINTS

- Contract artifacts: customer.xsd + CustomerService.wsdl.
- No live server required — design-time validation only.
- Feeds Lab 24, which implements the endpoint with Spring-WS.

TOOLS & TECHNOLOGIES

XML/WSDL Editor
Primary contract editor

xmllint (Validation)
or IDE schema validation

SoapUI
WSDL-driven testing (primary)

Postman / curl
Manual transport testing only

Maven is only needed if the project uses schema-validation plugins or generated-code tooling (e.g., wsimport, JAXB).

KEY TAKEAWAY This lab follows contract-first design: WSDL defines the contract, XSD defines the data — Lab 24 implements it with Spring-WS.

LAB TASKS AND DELIVERABLES

Follow a contract-first approach to design a SOAP contract for Northstar's CRM customer service.

#	TASK	DETAILS	FORMAT
1	Project Setup	Create lab13-crm layout; write operation-matrix.md.	Document
2	Design XSD Schema	customer.xsd: CustomerType, CustomerStatus, request/response elements.	.xsd
3	Design WSDL Contract	CustomerService.wsdl: messages, portType, binding, service.	.wsdl
4	Write Sample Envelopes	Create/Update/Get requests & responses for CUS-1001, CUS-1002.	.xml
5	Write Fault Samples	Not-found and validation fault envelopes.	.xml
6	Document & Handoff	soap-design-notes.md + partner handoff checklist.	Document

KEY TAKEAWAY This lab helps you design and document a partner-ready SOAP contract — Lab 24 implements it with Spring-WS.

LAB TASKS AND DELIVERABLES — EVALUATION CRITERIA

Follow a contract-first approach to design a SOAP contract for Northstar's CRM customer service.

EVALUATION CRITERIA — CONTRACT QUALITY

- Stable, versioned targetNamespace.
- elementFormDefault="qualified".
- No duplicate global element names.
- Reusable shared types (e.g., CustomerType).
- Document/literal wrapped binding.
- One request and one response element per operation.
- Typed SOAP Fault detail elements.
- Backward-compatible schema choices.

EVALUATION CRITERIA — FAULTS & VALIDATION

Business faults (not-found, invalid status, duplicate, validation) — defined in the WSDL.
Technical faults (unavailable, DB failure, server error) — generic fault policy is OK.
Valid endpoint address placeholder in <service>.
Validates successfully without network access.
CreateCustomer/UpdateCustomer/GetCustomer correctly defined.
Sample and fault envelopes are complete.

KEY TAKEAWAY This lab helps you design and document a partner-ready SOAP contract — Lab 24 implements it with Spring-WS.

MODULE SUMMARY — DEFINITION OF DONE

A SOAP contract is ready when it meets this Definition of Done -- contract-first, end to end.

1. Target namespace is stable and versioned.
2. XSD types reflect business meaning.
3. Optionality and cardinality are intentional.
4. Operations have explicit request and response elements.
5. Business faults are contractually defined.
6. WSDL uses interoperable binding conventions.
7. Sample envelopes validate against the contract.
8. Security requirements are documented.
9. Sensitive information is not logged.
10. Backward compatibility is reviewed.
11. Partner handoff documentation is complete.

KEY TAKEAWAY Use this checklist before publishing any SOAP contract — it turns module concepts into a concrete, reviewable definition of done.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of SOAP web services, WSDL, and contract-first design.

1. What's the primary purpose of SOAP?

A. Lightweight messaging over HTTP
B. Structured messaging using XML
C. Real-time event streaming
D. Defining RESTful APIs

2. Which WSDL component defines message data types?

A. <service>
B. <binding>
C. <types>
D. <portType>

3. What does XSD stand for?

A. XML Service Definition
B. XML Schema Definition
C. XML Structure Document
D. XML Style Descriptor

4. In contract-first approach, what's created first?

A. Client code
B. Database schema
C. Service implementation
D. Service contract (WSDL and XSD)

KEY TAKEAWAY SOAP is a structured XML messaging protocol — contract-first means the WSDL and XSD come before any code.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. Which namespace-qualified element is the root of a SOAP message?

A. SOAP Envelope (namespace-qualified)

B. <Message>

C. <Request>

D. <SOAPBody>

6. What does the SOAP Body contain?

A. HTTP headers

B. Authentication tokens

C. The actual request or response message

D. Routing information

7. How does a SOAP service indicate an error?

A. By returning HTTP 404

B. By throwing a Java exception only

C. By returning a SOAP Fault message

D. By closing the connection

8. Where is a SOAP processing error represented in the message?

A. As a SOAP Fault inside the SOAP Body

B. In the HTTP status code alone

C. In a custom HTTP header

D. It cannot be represented in the message

ANSWER KEY

- **1-B 2-C 3-B 4-D 5-A 6-C 7-C 8-A**

KEY TAKEAWAY SOAP delivers where security, reliability, compliance, and long-term stability matter most.

"Right technology, right problem, right environment."

MODULE 13 COMPLETE

SOAP API Design with Java

You can now design, implement, secure, and consume enterprise-grade SOAP web services with confidence.